

Ugly Number III Explanation

Absolutely. LeetCode 1201 — Ugly Number III is a very good problem for learning the combination of Binary Search + Inclusion-Exclusion + LCM.

The key is to stop thinking:

"How do I generate the ugly numbers?"

and instead think:

"For a given number x , how many ugly numbers are there from 1 to x ?"

Once we can answer that, binary search gives us the n th one.

1. Understand the problem

A number is ugly if it is divisible by at least one of:

a , b , c

For example:

$a = 2$
 $b = 3$
 $c = 5$

Numbers from 1 to 10:

1	2	3	4	5	6	7	8	9	10
	✓	✓	✓	✓	✓		✓	✓	✓

So ugly numbers are:

2, 3, 4, 5, 6, 8, 9, 10...

We need the n th one.

2. Why can't we simply generate them?

Suppose:

$n = 1,000,000,000$

We obviously cannot check numbers one by one.

Even if we check:

```
for (long long x = 1; ; x++)
```

it can take far too long.

So we need a way to count ugly numbers quickly.

3. The most important observation

Suppose I ask:

How many numbers from 1 to x are divisible by a ?

The answer is:

x / a

For example:

$x = 20$
 $a = 3$

Multiples of 3:

3, 6, 9, 12, 15, 18

There are:

$$20 / 3 = 6$$

So:

$$x / a$$

counts numbers divisible by a .

Similarly:

$$x / b$$

$$x / c$$

4. First attempt: Add them

We might think:

$$\text{count} = x/a + x/b + x/c;$$

But there's a problem.

Consider:

$$a = 2$$

$$b = 3$$

$$x = 6$$

Multiples of 2:

$$2, 4, 6$$

Multiples of 3:

$$3, 6$$

If we add:

$$3 + 2 = 5$$

But the actual ugly numbers are:

$$2, 3, 4, 6$$

Only 4 numbers.

Why?

Because 6 was counted twice.

5. Inclusion-Exclusion

This is where Inclusion-Exclusion Principle comes in.

For two sets:

$$|A \cup B| = |A| + |B| - |A \cap B|$$

For three sets:

$$\begin{aligned} &|A \cup B \cup C| \\ &= |A| + |B| + |C| \\ &\quad - |A \cap B| \\ &\quad - |B \cap C| \\ &\quad - |A \cap C| \\ &\quad + |A \cap B \cap C| \end{aligned}$$

That's exactly what we need.

6. What does intersection mean here?

Suppose:

$$a = 2$$

$$b = 3$$

A number divisible by both 2 and 3 must be divisible by:

```
LCM(2, 3) = 6
```

So:

```
x / lcm(a, b)
```

counts numbers divisible by both a and b.

Similarly:

```
x / lcm(a, c)
x / lcm(b, c)
```

And numbers divisible by all three are divisible by:

```
LCM(a, b, c)
```

7. Therefore our counting function is

For any x:

```
count(x) =
  x/a
+ x/b
+ x/c
- x/lcm(a,b)
- x/lcm(a,c)
- x/lcm(b,c)
+ x/lcm(a,b,c)
```

This tells us:

```
How many ugly numbers are ≤ x?
```

This is the heart of the problem.

8. Let's understand with an example

Take:

```
a = 2
b = 3
c = 5
x = 10
```

First:

```
10 / 2 = 5
10 / 3 = 3
10 / 5 = 2
```

So:

```
5 + 3 + 2 = 10
```

But duplicates exist.

Calculate LCMs:

```
lcm(2,3) = 6
lcm(2,5) = 10
lcm(3,5) = 15
```

Subtract:

```
10/6 = 1
10/10 = 1
10/15 = 0
```

So:

```
10 - 1 - 1 - 0 = 8
```

Now:

```
lcm(2,3,5) = 30
10/30 = 0
```

Therefore:

```
count(10) = 8
```

And indeed the ugly numbers ≤ 10 are:

```
2, 3, 4, 5, 6, 8, 9, 10
```

Exactly 8.

9. Now the Binary Search idea

We know how to answer:

```
How many ugly numbers are  $\leq x$ ?
```

Let's call this:

```
count(x)
```

We want the smallest x such that:

```
count(x) >= n
```

Why?

Because if there are at least n ugly numbers up to x , then the n th ugly number is somewhere at or before x .

10. Example

Suppose:

```
n = 5
a = 2
b = 11
c = 13
```

Ugly numbers:

```
2, 4, 6, 8, 10, 11, 12, 13...
```

We want:

```
5th = 10
```

Consider:

```
x = 9
```

Ugly numbers ≤ 9 :

```
2, 4, 6, 8
```

So:

```
count(9) = 4
```

Since:

```
4 < 5
```

the answer must be greater than 9.

Now:

```
x = 10
```

Ugly numbers:

```
2, 4, 6, 8, 10
```

So:

```
count(10) = 5
```

Therefore the answer is:

```
10
```

11. Why Binary Search works

The counting function is monotonic.

For example:

x	count(x)
1	0
2	1
3	1
4	2
5	2
6	3
7	3
8	4
9	4
10	5
11	6
...	

Notice:

```
count(x)
```

can stay the same or increase, but it can never decrease.

Therefore we can binary search.

12. Binary Search condition

We want:

```
smallest x where count(x) >= n
```

So:

```
if (count(mid) >= n)
{
    // mid could be the answer
    right = mid;
}
else
{
    // answer must be bigger
    left = mid + 1;
}
```

This is a classic:

Find First True

binary search.

13. What should the search range be?

The smallest ugly number is at least:

```
min(a, b, c)
```

But we can simply use:

```
left = 1
```

For the upper bound, the n th ugly number cannot be greater than:

```
n * min(a,b,c)
```

Because every multiple of the smallest value is ugly.

For example:

```
a = 2
b = 3
c = 5
n = 10
```

There are already 10 multiples of 2:

```
2,4,6,8,10,12,14,16,18,20
```

Therefore the answer is at most:

```
10 * 2 = 20
```

So:

```
right = 1LL * n * min({a,b,c});
```

14. Why long long?

This is very important.

n, a, b, c can be large.

When we calculate:

```
n * min(a,b,c)
```

it can exceed the range of a normal `int`.

So use:

```
long long
```

throughout the calculations.

15. Calculating LCM

We know:

```
LCM(a,b) = (a / GCD(a,b)) * b
```

Do division first to reduce the chance of overflow:

```
long long lcm(long long a, long long b)
{
    return a / gcd(a, b) * b;
}
```

C++17 has:

```
std::gcd()
```

from:

```
#include <numeric>
```

16. Three-way LCM

Instead of writing a special formula, we can do:

```
long long ab = lcm(a, b);
long long ac = lcm(a, c);
long long bc = lcm(b, c);

long long abc = lcm(ab, c);
```

So:

```
LCM(a,b,c)
=
LCM(LCM(a,b),c)
```

17. Complete counting function

```
long long countUgly(long long x,
                    long long a,
                    long long b,
                    long long c)
{
    long long ab = lcm(a, b);
    long long ac = lcm(a, c);
    long long bc = lcm(b, c);

    long long abc = lcm(ab, c);

    return x / a
        + x / b
        + x / c
        - x / ab
        - x / ac
        - x / bc
        + x / abc;
}
```

18. Complete solution

```
class Solution {
public:

    long long lcm(long long a, long long b) {
        return a / gcd(a, b) * b;
    }

    long long countUgly(long long x,
                        long long a,
                        long long b,
                        long long c) {

        long long ab = lcm(a, b);
        long long ac = lcm(a, c);
        long long bc = lcm(b, c);

        long long abc = lcm(ab, c);

        return x / a
            + x / b
            + x / c
            - x / ab
            - x / ac
            - x / bc
            + x / abc;
    }

    int nthUglyNumber(int n, int a, int b, int c) {

        long long left = 1;

        long long right =
            1LL * n * min({a, b, c});

        while (left < right) {

            long long mid =
                left + (right - left) / 2;

            if (countUgly(mid, a, b, c) >= n) {
                right = mid;
            }
            else {
                left = mid + 1;
            }
        }
    }
};
```

```
        }  
    }  
  
    return left;  
}  
};
```

19. Dry Run of Binary Search

Let's take:

```
n = 3  
a = 2  
b = 3  
c = 5
```

Search range:

```
left = 1  
right = 3 * min(2,3,5)  
        = 6
```

So:

```
[1, 6]
```

Iteration 1

```
mid = 3
```

Count ugly numbers ≤ 3 :

```
2, 3
```

Therefore:

```
count(3) = 2
```

We need:

```
3
```

Since:

```
2 < 3
```

answer must be larger.

So:

```
left = 4
```

Range:

```
[4, 6]
```

Iteration 2

```
mid = 5
```

Ugly numbers ≤ 5 :

```
2, 3, 4, 5
```

Count:

```
4
```

Now:

```
4 >= 3
```

So 5 could be the answer.

Move left:

```
right = 5
```

Range:

```
[4,5]
```

Iteration 3

```
mid = 4
```

Ugly numbers ≤ 4 :

```
2, 3, 4
```

Count:

```
3
```

Since:

```
3 >= 3
```

4 could be the answer.

So:

```
right = 4
```

Now:

```
left = 4  
right = 4
```

Stop.

Answer:

```
4
```

20. The complete thought process

This problem can be remembered as this chain:

```
Need nth ugly number  
↓  
Can't generate numbers one-by-one  
↓  
Can we count ugly numbers  $\leq x$ ?  
↓  
Numbers divisible by a =  $x/a$   
Numbers divisible by b =  $x/b$   
Numbers divisible by c =  $x/c$   
↓  
Duplicates exist  
↓  
Use Inclusion-Exclusion  
↓  
Intersection means divisible by LCM  
↓  
Now count(x) is O(1)  
↓  
count(x) is monotonic  
↓  
Binary Search  
↓  
Find smallest x where count(x)  $\geq n$ 
```

This is the main pattern you should learn from this problem.

21. Complexity

There are only a constant number of operations in:

```
countUgly()
```

So approximately:

```
O(1)
```

per binary-search iteration.

Binary search takes:

```
O(log(n * min(a,b,c)))
```

Therefore:

Time

```
O(log(n * min(a,b,c)))
```

Space

```
O(1)
```

22. The important concepts to learn

This one problem combines three important DSA concepts:

① Binary Search on Answer

Instead of searching for an element in an array:

```
Find smallest x satisfying some condition
```

Here:

```
countUgly(x) >= n
```

② Inclusion-Exclusion

When counting the union of multiple sets:

```
A ∪ B ∪ C
```

use:

```
A + B + C
- AB - AC - BC
+ ABC
```

③ LCM

When asking:

```
"How many numbers are divisible by both a and b?"
```

the answer is:

```
x / LCM(a,b)
```

because a number divisible by both must be divisible by their LCM.

One-line formula to memorize

For this problem:

```
count(x)
=
x/a + x/b + x/c
- x/lcm(a,b)
- x/lcm(a,c)
- x/lcm(b,c)
+ x/lcm(a,b,c)
```

Then:

Binary Search for the smallest x
such that

`count(x) >= n`

That's the entire solution.

If you're preparing for interviews/CP, the bigger takeaway is: whenever a problem asks for the k -th/smallest value and you can efficiently answer "how many values are $\leq X$ ", immediately consider Binary Search on Answer.